

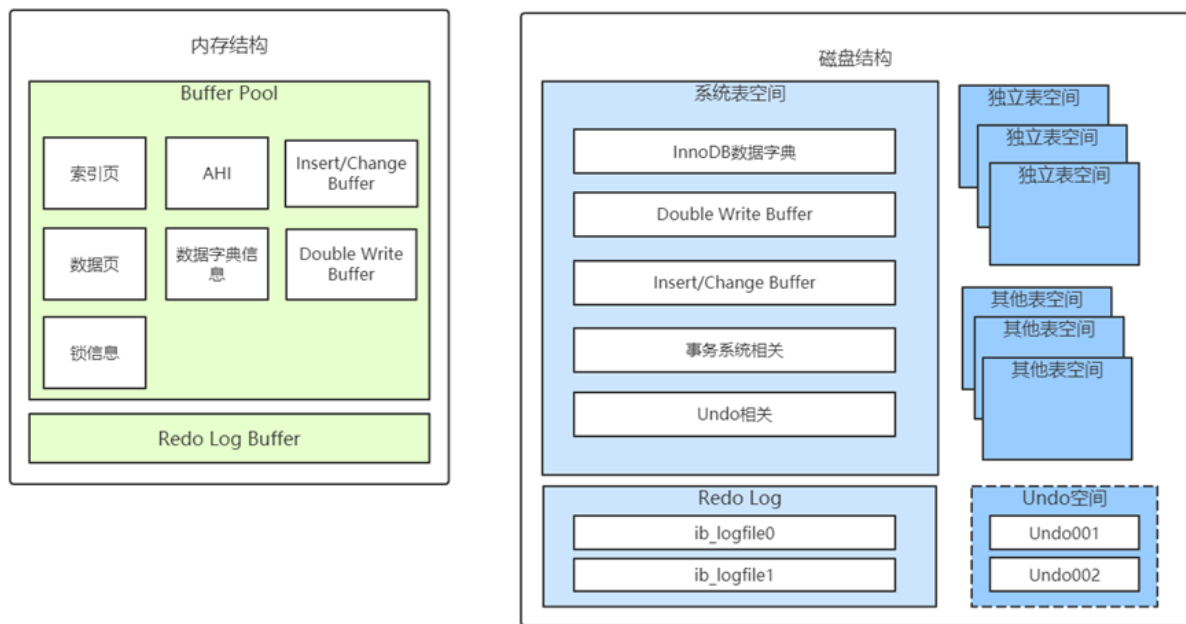
1.InnoDB引擎底层解析

InnoDB的三大特性：

- 双写机制
- Buffer Pool
- 自适应Hash索引

自适应Hash索引在之前的索引课中已经讲到了，这节课不再做陈述。同时我们对InnoDB不能只是光看亮点，还是要体系化的去学习。

InnoDB的内存结构和磁盘存储结构图总结如下：



看这种结构图大家肯定是比较晕的，所以我们用需求来驱动进行讲解。

- 1、InnoDB对于我们来说还是一个黑盒，我们只负责使用客户端发送请求并等待服务器返回结果，表中的数据到底存到了哪里？
- 2、表中的数据以什么格式存放的？
- 3、InnoDB是以什么方式来访问的这些数据？
- 4、InnoDB中的事务、锁等的原理是怎样？

1.1.InnoDB记录存储结构和索引页结构

InnoDB是一个将表中的数据存储在磁盘上的存储引擎，所以即使关机后重启我们的数据还是存在的。而真正处理数据的过程是发生在内存中的，所以需要把磁盘中的数据加载到内存中，如果是处理写入或修改请求的话，还需要把内存中的内容刷新到磁盘上。而我们知道读写磁盘的速度非常慢，和内存读写差了几个数量级，所以当我们想从表中获取某些记录时，InnoDB存储引擎需要一条一条的把记录从磁盘上读出来？

InnoDB采取的方式是：将数据划分为若干个页，以页作为磁盘和内存之间交互的基本单位，InnoDB中页的大小一般为 16 KB。也就是在一般情况下，一次最少从磁盘中读取16KB的内容到内存中，一次最少把内存中的16KB内容刷新到磁盘中。

我们平时是以记录为单位来向表中插入数据的，这些记录在磁盘上的存放方式也被称为**行格式**。

1.1.1.行格式

InnoDB存储引擎设计了4种不同类型的行格式，分别是Compact、Redundant、Dynamic和Compressed行格式。我们可以查看默认值：

```
show variables like 'innodb_default_row_format'
```

```
mysql> show variables like 'innodb default row format';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_default_row_format | dynamic |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

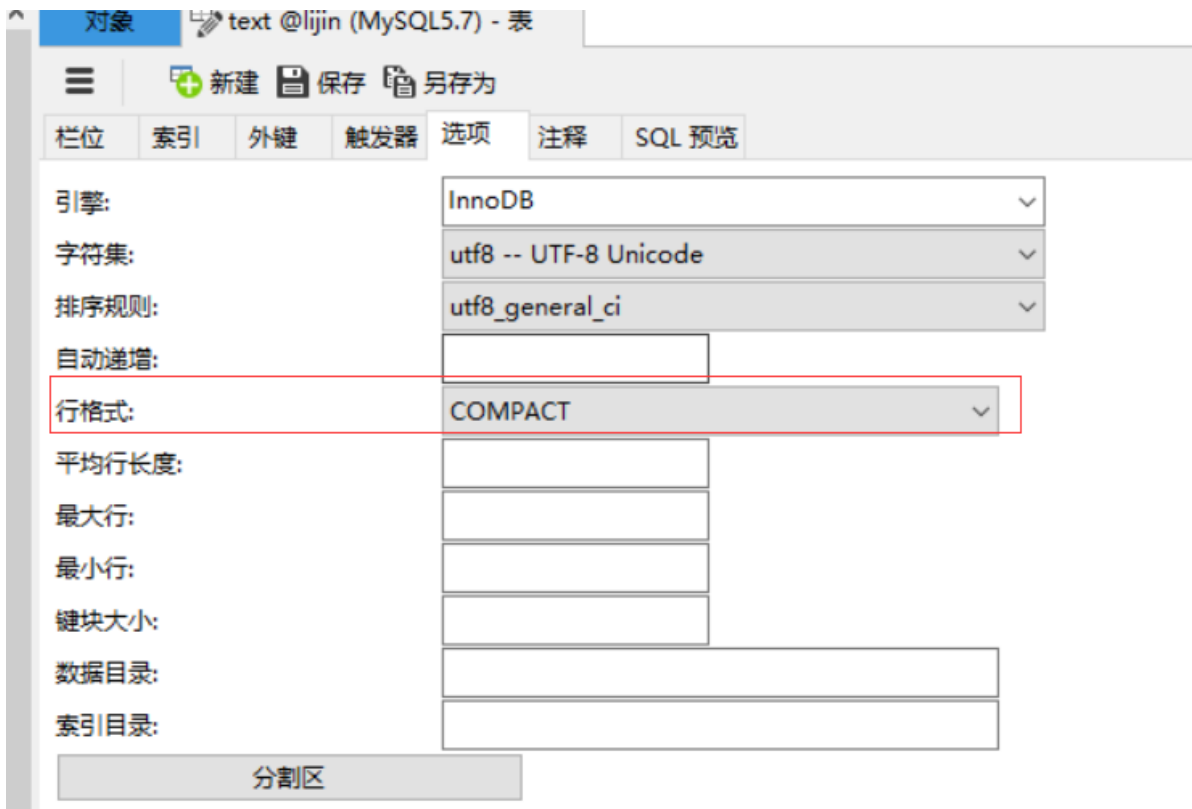
The image shows the MySQL DDL Wizard interface for creating a table. The 'Engine' (引擎) is set to 'InnoDB'. The 'Character Set' (字符集) is 'utf8 -- UTF-8 Unicode'. The 'Collation' (排序规则) is 'utf8_general_ci'. The 'Row Format' (行格式) is set to 'DYNAMIC', which is highlighted with a red box. Other settings like 'Auto Increment' (自动递增), 'Average Row Length' (平均行长度), 'Maximum Rows' (最大行), 'Minimum Rows' (最小行), 'Block Size' (块大小), 'Data Directory' (数据目录), and 'Index Directory' (索引目录) are shown as empty input fields. At the bottom, there is a 'Partitioning' (分区) section.

我们可以在创建或修改表的语句中指定行格式：

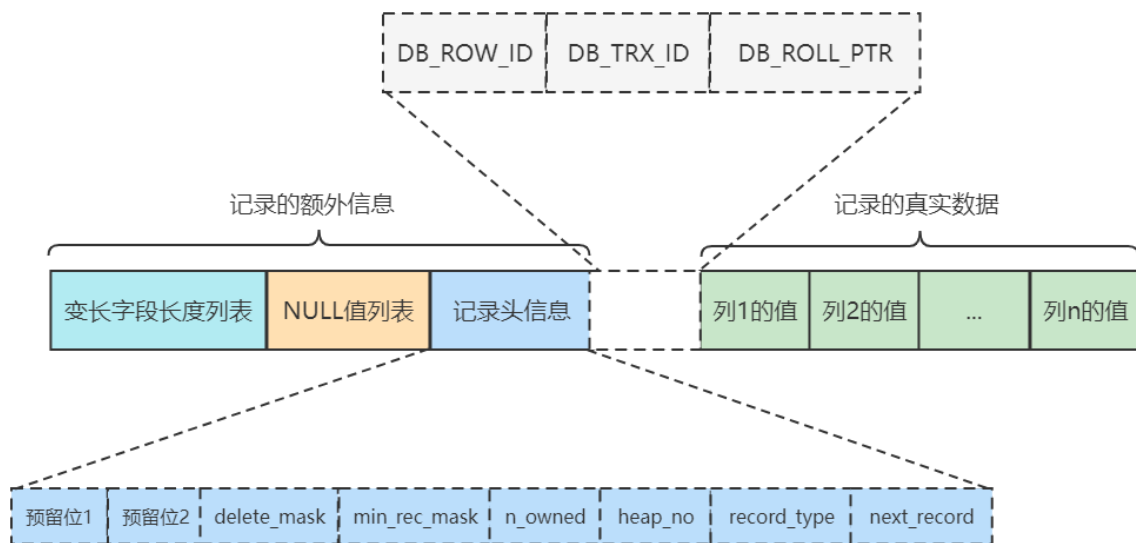
```
CREATE TABLE 表名 (列的信息) ROW_FORMAT=行格式名称
```

1.1.1.1.COMPACT

```
create table text(c1 VARCHAR(10)) ROW_FORMAT=COMPACT;
```



COMPACT行格式示意图如下：



变长字段长度列表

我们知道MySQL支持一些变长的数据类型，比如VARCHAR(M)、VARBINARY(M)、各种TEXT类型，各种BLOB类型，我们也可以把拥有这些数据类型的列称为变长字段，变长字段中存储多少字节的数据是不固定的，所以我们在存储真实数据的时候需要顺便把这些数据占用的字节数也存起来。如果该可变字段允许存储的最大字节数（ $M \times W$ ）超过255字节并且真实存储的字节数（L）超过127字节，则使用2个字节，否则使用1个字节。

NULL值列表

表中的某些列可能存储NULL值，如果把这些NULL值都放到记录的真实数据中存储会很占地方，所以Compact行格式把这些值为NULL的列统一管理起来，存储到NULL值列表。每个允许存储NULL的列对应一个二进制位，二进制位的值为1时，代表该列的值为NULL。二进制位的值为0时，代表该列的值不为NULL。

记录头信息

它是由固定的5个字节组成。5个字节也就是40个二进制位，不同的位代表不同的意思。

	二进制位数	解释
预留位1	1	没有使用
预留位2	1	没有使用
delete_mask	1	标记该记录是否被删除
min_rec_mask	1	B+树的每层非叶子节点中的最小记录都会添加该标记
n_owned	4	表示当前记录拥有的记录数
heap_no	13	表示当前记录在页的位置信息
record_type	3	表示当前记录的类型， 0表示普通记录， 1表示B+树非叶子节点记录， 2表示最小记录， 3表示最大记录
next_record	16	表示下一条记录的相对位置

隐藏列信息

MySQL会为每个记录默认的添加一些列（也称为隐藏列），包括：

DB_ROW_ID(row_id)：非必须，6字节，表示行ID，唯一标识一条记录

InnoDB表对主键的生成策略是：优先使用用户自定义主键作为主键，如果用户没有定义主键，则选取一个Unique键作为主键，如果表中连Unique键都没有定义的话，则InnoDB会为表默认添加一个名为row_id的隐藏列作为主键。

DB_TRX_ID：必须，6字节，表示事务ID

DB_ROLL_PTR：必须，7字节，表示回滚指

其他的行格式和Compact行格式差别不大。

1.1.1.2.Redundant行格式

Redundant行格式是MySQL5.0之前用的一种行格式，不予深究。

1.1.1.3.Dynamic和Compressed行格式

MySQL5.7的默认行格式就是Dynamic，Dynamic和Compressed行格式和Compact行格式挺像，只不过在处理行溢出数据时有所不同。Compressed行格式和Dynamic不同的一点是，Compressed行格式会采用压缩算法对页面进行压缩，以节省空间。

1.1.1.4. 数据溢出

如果我们定义一个表，表中只有一个VARCHAR字段，如下：

```
CREATE TABLE test_varchar( c VARCHAR(60000) )
```

然后往这个字段插入60000个字符，会发生什么？

前边说过，MySQL中磁盘和内存交互的基本单位是页，也就是说MySQL是以页为基本单位来管理存储空间的，我们的记录都会被分配到某个页中存储。而一个页的大小一般是16KB，也就是16384字节，而一个VARCHAR(M)类型的列就最多可以存储65532个字节，这样就可能造成一个页存放不了一条记录的情况。

在Compact和Redundant行格式中，对于占用存储空间非常大的列，在记录的真实数据处只会存储该列的该列的前768个字节的数据，然后把剩余的数据分散存储在几个其他的页中，记录的真实数据处用20个字节存储指向这些页的地址。这个过程也叫做行溢出，存储超出768字节的那些页面也被称为溢出页。

Dynamic和Compressed行格式，不会在记录的真实数据处存储字段真实数据的前768个字节，而是把所有的字节都存储到其他页面中，只在记录的真实数据处存储其他页面的地址。

1.1.2.索引页格式

前边我们简单提了一下页的概念，它是InnoDB管理存储空间的基本单位，一个页的大小一般是16KB。

InnoDB为了不同的目的而设计了许多种不同类型的页，存放我们表中记录的那种类型的页自然也是其中的一员，官方称这种存放记录的页为索引（INDEX）页，不过要理解成数据页也没问题，毕竟存在着聚簇索引这种索引和数据混合的东西。

1.1.2.1.数据页结构

一个InnoDB数据页的存储空间大致被划分成了7个部分：



name	名称	长度	备注
File Header	文件头部	38字节	页的一些通用信息
Page Header	页面头部	56字节	数据页专有的一些信息
Infimum + Supremum	最小记录和最大记录	26字节	两个虚拟的行记录
User Records	用户记录	大小不确定	实际存储的行记录内容
Free Space	空闲空间	大小不确定	页中尚未使用的空间
Page Directory	页面目录	大小不确定	页中的某些记录的相对位置
File Trailer	文件尾部	8字节	校验页是否完整

User Records

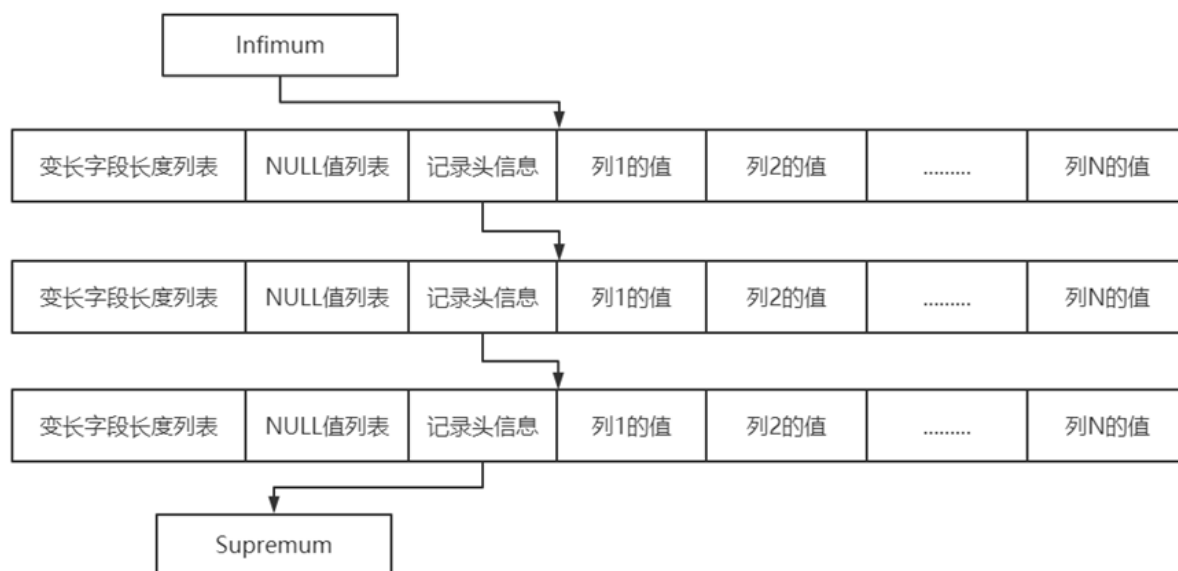
我们自己存储的记录会按照我们指定的行格式存储到User Records部分。但是在一开始生成页的时候，其实并没有User Records这个部分，每当我们插入一条记录，都会从Free Space部分，也就是尚未使用的存储空间中申请一个记录大小的空间划分到User Records部分，当Free Space部分的空间全部被User Records部分替代掉之后，也就意味着这个页使用完了，如果还有新的记录插入的话，就需要去申请新的页了。

当前记录被删除时，则会修改记录头信息中的delete_mask为1，也就是说被删除的记录还在页中，还在真实的磁盘上。这些被删除的记录之所以不立即从磁盘上移除，是因为移除它们之后把其他的记录在磁盘上重新排列需要性能消耗。

所以只是打一个删除标记而已，所有被删除掉的记录都会组成一个所谓的垃圾链表，在这个链表中的记录占用的空间称之为所谓的可重用空间，之后如果有新记录插入到表中的话，可能把这些被删除的记录占用的存储空间覆盖掉。

同时我们插入的记录在会记录自己在本页中的位置，写入了记录头信息中heap_no部分。heap_no值为0和1的记录是InnoDB自动给每个页增加的两个记录，称为伪记录或者虚拟记录。这两个伪记录一个代表最小记录，一个代表最大记录，这两条存放在页的User Records部分，他们被单独放在一个称为Infimum + Supremum的部分。

记录头信息中next_record记录了从当前记录的真实数据到下一条记录的真实数据的地址偏移量。这其实是个链表，可以通过一条记录找到它的下一条记录。但是需要注意注意再注意的一点是，下一条记录指得并不是按照我们插入顺序的下一条记录，而是按照主键值由小到大的顺序的下一条记录。而且规定Infimum记录（也就是最小记录）的下一条记录就是本页中主键值最小的用户记录，而本页中主键值最大的用户记录的下一条记录就是Supremum记录（也就是最大记录）



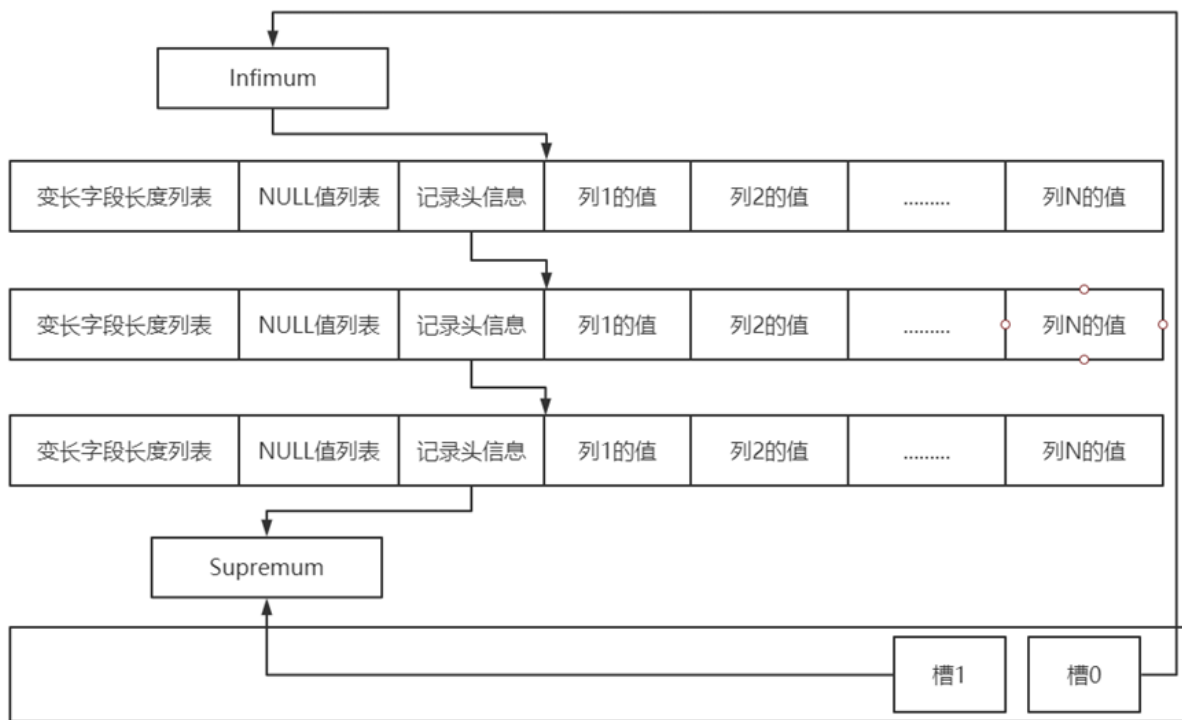
我们的记录按照主键从小到大的顺序形成了一个单链表，记录被删除，则从这个链表上摘除。

Page Directory

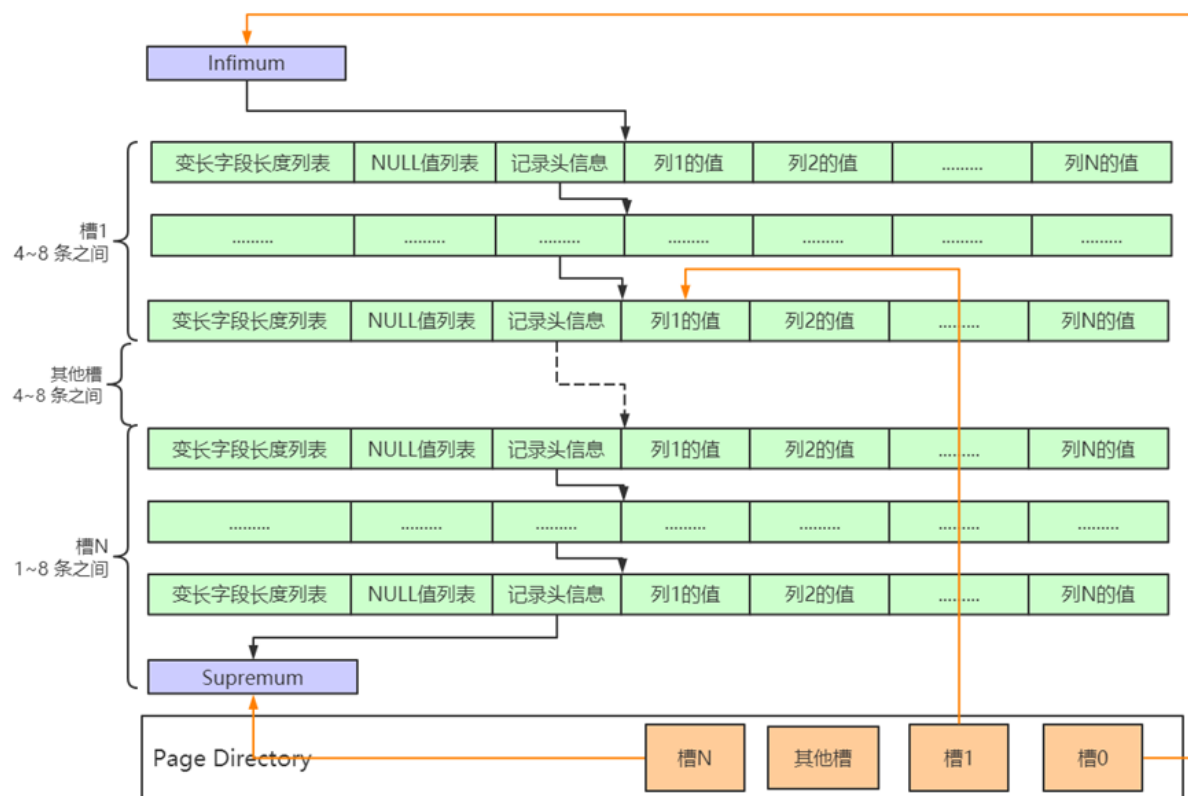
Page Directory主要是解决记录链表的查找问题。如果我们想根据主键值查找页中的某条记录该咋办？按链表查找的办法：从Infimum记录（最小记录）开始，沿着链表一直往后找，总会找到或者找不到。

InnoDB的改进是，为页中的记录再制作了一个目录，他们的制作过程是这样的：

- 1、将所有正常的记录（包括最大和最小记录，不包括标记为已删除的记录）划分为几个组。
- 2、每个组的最后一条记录（也就是组内最大的那条记录）的头信息中的n_owned属性表示该记录拥有多少条记录，也就是该组内共有几条记录。
- 3、将每个组的最后一条记录的地址偏移量单独提取出来按顺序存储到靠近页的尾部的地方，这个地方就是所谓的Page Directory，也就是页目录页面目录中的这些地址偏移量被称为槽（英文名：Slot），所以这个页面目录就是由槽组成的。



4、每个分组中的记录条数是有规定的：对于最小记录所在的分组只能有 1 条记录，最大记录所在的分组拥有的记录条数只能在1到8 条之间，剩下的分组中记录的条数范围只能是在 4到8 条之间，如下图：



这样，一个数据页中查找指定主键值的记录的过程分为两步：

通过二分法确定该记录所在的槽，并找到该槽所在分组中主键值最小的那条记录。

通过记录的next_record属性遍历该槽所在的组中的各个记录。

Page Header

InnoDB为了能得到一个数据页中存储的记录的状态信息，比如本页中已经存储了多少条记录，第一条记录的地址是什么，页目录中存储了多少个槽等等，特意在页中定义了一个叫Page Header的部分，它是页结构的第二部分，这个部分占用固定的56个字节，专门存储各种状态信息。

File Header

File Header针对各种类型的页都通用，也就是说不同类型的页都会以File Header作为第一个组成部分，它描述了一些针对各种页都通用的一些信息，比方说页的类型，这个页的编号是多少，它的上一个页、下一个页是谁，页的校验和等等，这个部分占用固定的38个字节。

页的类型，包括Undo日志页、段信息节点、InsertBuffer空闲列表、Insert Buffer位图、系统页、事务系统数据、表空间头部信息、扩展描述页、溢出页、索引页，有些页会在后面的课程看到。

同时通过上一个页、下一个页建立一个双向链表把许许多多的页就串联起来，而无需这些页在物理上真正连着。但是并不是所有类型的页都有上一个和下一个页的属性，数据页是有这两个属性的，所以所有的数据页其实是一个双向链表。

File Trailer

我们知道InnoDB存储引擎会把数据存储到磁盘上，但是磁盘速度太慢，需要以页为单位把数据加载到内存中处理，如果该页中的数据在内存中被修改了，那么在修改后的某个时间需要把数据同步到磁盘中。但是在同步了一半的时候中断电了咋办？

为了检测一个页是否完整（也就是在同步的时候有没有发生只同步一半的尴尬情况），InnoDB每个页的尾部都加了一个File Trailer部分，这个部分由8个字节组成，可以分成2个小部分：

前4个字节代表页的校验和

这个部分是和File Header中的校验和相对应的。每当一个页面在内存中修改了，在同步之前就要把它的校验和算出来，因为File Header在页面的前边，所以校验和会被首先同步到磁盘，当完全写完时，校验和也会被写到页的尾部，如果完全同步成功，则页的首部和尾部的校验和应该是一致的。如果写了一半儿断电了，那么在File Header中的校验和就代表着已经修改过的页，而在File Trailer中的校验和代表着原先的页，二者不同则意味着同步中间出了错。

后4个字节代表页面被最后修改时对应的日志序列位置（LSN），这个也和校验页的完整性有关。

这个File Trailer与File Header类似，都是所有类型的页通用的。

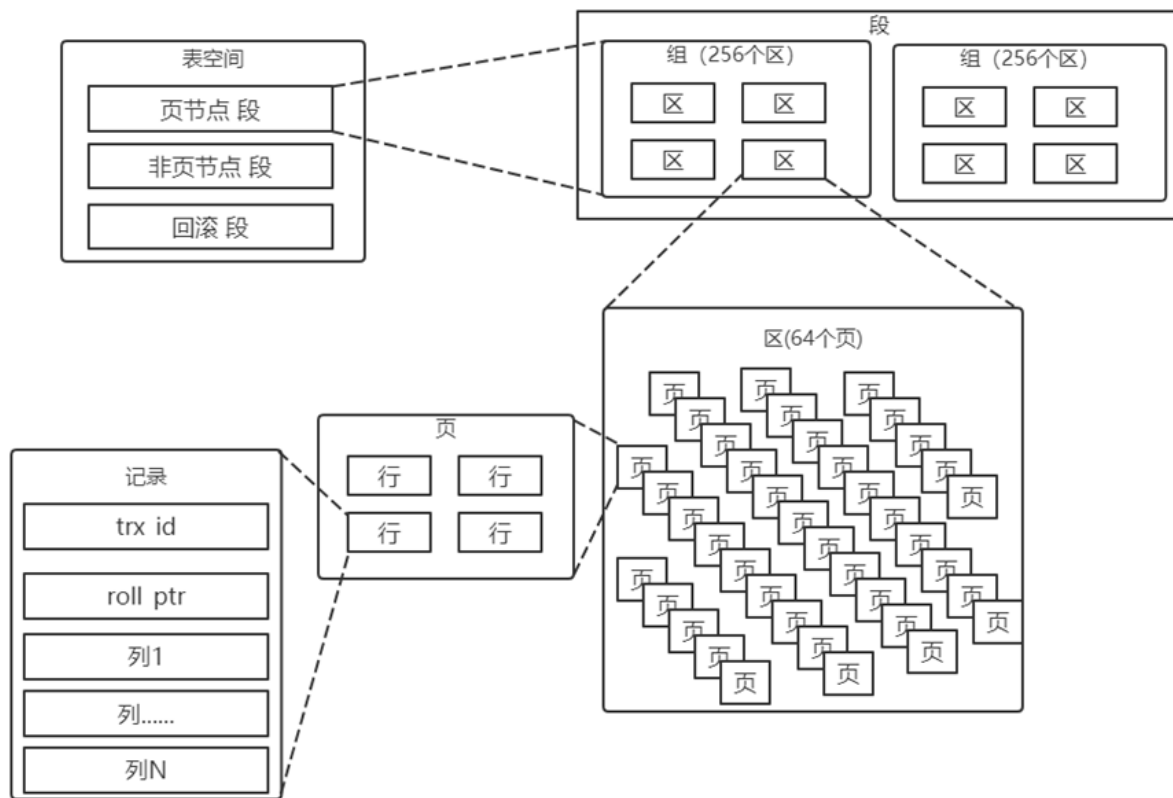
1.2.InnoDB的表空间

表空间是一个抽象的概念，对于系统表空间来说，对应着文件系统中一个或多个实际文件；对于每个独立表空间来说，对应着文件系统中一个名为表名.ibd的实际文件。大家可以把表空间想象成被切分为许许多多多个页的池子，当我们想为某个表插入一条记录的时候，就从池子中捞出一个对应的页来把数据写进去。

再回忆一次，InnoDB是以页为单位管理存储空间的，我们的聚簇索引（也就是完整的表数据）和其他的二级索引都是以B+树的形式保存到表空间的，而B+树的节点就是数据页。

任何类型的页都有File Header这个部分，File Header中专门的地方

（FIL_PAGE_ARCH_LOG_NO_OR_SPACE_ID）保存页属于哪个表空间，同时表空间中的每一个页都对应着一个页号（FIL_PAGE_OFFSET），这个页号由4个字节组成，也就是32个比特位，所以一个表空间最多可以拥有 2^{32} 个页，如果按照页的默认大小16KB来算，一个表空间最多支持64TB的数据。



1.2.1.独立表空间结构

1.2.1.1. 区 (extent)

表空间中的页可以达到 2^{32} 个页，实在是太多了，为了更好的管理这些页面，InnoDB中还有一个区（英文名：extent）的概念。对于16KB的页来说，连续的64个页就是一个区，也就是说一个区默认占用1MB空间大小。

不论是系统表空间还是独立表空间，都可以看成是由若干个区组成的，每256个区又被划分成一个组。

第一个组最开始的3个页面的类型是固定的：用来登记整个表空间的一些整体属性以及本组所有的区被称为FSP_HDR，也就是extent 0 ~ extent 255这256个区，整个表空间只有一个FSP_HDR。

其余各组最开始的2个页面的类型是固定的，一个XDES类型，用来登记本组256个区的属性，FSP_HDR类型的页面其实和XDES类型的页面的作用类似，只不过FSP_HDR类型的页面还会额外存储一些表空间的属性。

引入区的主要目的是什么？

我们每向表中插入一条记录，本质上就是向该表的聚簇索引以及所有二级索引代表的B+树的节点中插入数据。而B+树的每一层中的页都会形成一个双向链表，如果是以页为单位来分配存储空间的话，双向链表相邻的两个页之间的物理位置可能离得非常远。

我们介绍B+树索引的适用场景的时候特别提到范围查询只需要定位到最左边的记录和最右边的记录，然后沿着双向链表一直扫描就可以了，而如果链表中相邻的两个页物理位置离得非常远，就是所谓的随机I/O。再一次强调，磁盘的速度和内存的速度差了好几个数量级，随机I/O是非常慢的，所以我们应该尽量让链表中相邻的页的物理位置也相邻，这样进行范围查询的时候才可以使用所谓的顺序I/O。

一个区就是在物理位置上连续的64个页。在表中数据量大的时候，为某个索引分配空间的时候就不再按照页为单位分配了，而是按照区为单位分配，甚至在表中的数据十分非常特别多的时候，可以一次性分配多个连续的区，从性能角度看，可以消除很多的随机I/O。

1.2.1.2. 段 (segment)

我们提到的范围查询，其实是对B+树叶子节点中的记录进行顺序扫描，而如果不区分叶子节点和非叶子节点，统统把节点代表的页面放到申请到的区中的话，进行范围扫描的效果就大打折扣了。所以InnoDB对B+树的叶子节点和非叶子节点进行了区别对待，也就是说叶子节点有自己独有的区，非叶子节点也有自己独有的区。

存放叶子节点的区的集合就算是一个段 (segment)，存放非叶子节点的区的集合也算是一个段。也就是说一个索引会生成2个段，一个叶子节点段，一个非叶子节点段。

段其实不对应表空间中某一个连续的物理区域，而是一个逻辑上的概念。

1.2.2. 系统表空间

1.2.2.1. 整体结构

系统表空间的结构和独立表空间基本类似，只不过由于整个MySQL进程只有一个系统表空间，在系统表空间中会额外记录一些有关整个系统信息的页面，所以会比独立表空间多出一些记录这些信息的页面，相当于是表空间之首，所以它的表空间 ID (Space ID) 是0。

系统表空间的extent 1和extent 2这两个区，也就是页号从64~191这128个页面被称为Doublewrite buffer，也就是双写缓冲区。

1.2.2.2. 双写缓冲区/双写机制

双写缓冲区/双写机制是InnoDB的三大特性之一，还有两个是Buffer Pool、自适应Hash索引。

它是一种特殊文件flush技术，带给InnoDB存储引擎的是数据页的可靠性。它的作用是，在把页写到数据文件之前，InnoDB先把它们写到一个叫doublewrite buffer (双写缓冲区) 的连续区域内，在写doublewrite buffer完成后，InnoDB才会把页写到数据文件的适当的位置。如果在写页的过程中发生意外崩溃，InnoDB在稍后的恢复过程中在doublewrite buffer中找到完好的page副本用于恢复。

doublewrite buffer是InnoDB在系统表空间上的128个页 (2个区，extend1和extend2)，大小是2MB

所以在正常的情况下，MySQL写数据页时，会写两遍到磁盘上，第一遍是写到doublewrite buffer，第二遍是写到真正的数据文件中。如果发生了极端情况 (断电)，InnoDB再次启动后，发现了一个页数据已经损坏，那么此时就可以从doublewrite buffer中进行数据恢复了。

所以，虽然叫双写缓冲区，但是这个缓冲区不仅在内存中有，更多的是属于MySQL的系统表空间，属于磁盘文件的一部分。那为什么要引入一个双写机制呢？

InnoDB的页大小一般是16KB，其数据校验也是针对这16KB来计算的，将数据写入到磁盘是以页为单位进行操作的。而操作系统写文件是以4KB作为单位的，那么每写一个InnoDB的页到磁盘上，操作系统需要写4个块。

而计算机硬件和操作系统，在极端情况下 (比如断电) 往往并不能保证这一操作的原子性，16K的数据，写入4K时，发生了系统断电或系统崩溃，只有一部分写是成功的，这种情况下会产生partial page write (部分页写入) 问题。这时页数据出现不一样的情形，从而形成一个"断裂"的页，使数据产生混乱。在InnoDB存储引擎未使用doublewrite技术前，曾经出现过因为部分写失效而导致数据丢失的情况。

doublewrite是在一个连续的存储空间，所以硬盘在写数据的时候是顺序写，而不是随机写，这样性能影响不大，相比不双写，降低了大概5-10%左右。

所以，在一些情况下可以关闭doublewrite以获取更高的性能。比如在slave上可以关闭，因为即使出现了partial page write问题，数据还是可以从中继日志中恢复。比如某些文件系统ZFS本身有些文件系统本身就提供了部分写失效的防范机制，也可以关闭。

在数据库异常关闭的情况下启动时，都会做数据库恢复（redo）操作，恢复的过程中，数据库都会检查页面是不是合法（校验等等），如果发现一个页面校验结果不一致，则此时会用到双写这个功能。

有经验的同学也许会想到，如果发生写失效，可以通过重做日志(Redo Log)进行恢复啊！但是要注意，重做日志中记录的是对页的物理操作，如偏移量800,写'aaaa'记录，而不是页面的全量记录，而如果发生partial page write（部分页写入）问题时，出现问题的是未修改过的数据，此时重做日志(Redo Log)无能为力。写doublewrite buffer成功了，这个问题就不用担心了。

1.2.2.2.InnoDB数据字典(Data Dictionary Header)

我们平时使用INSERT语句向表中插入的那些记录称之为用户数据，MySQL只是作为一个软件来为我们来保管这些数据，提供方便的增删改查接口而已。但是每当我们向一个表中插入一条记录的时候，MySQL先要校验一下插入语句对应的表存不存在，插入的列和表中的列是否符合，如果语法没有问题的话，还需要知道该表的聚簇索引和所有二级索引对应的根页面是哪个表空间的哪个页面，然后把记录插入对应索引的B+树中。所以说，MySQL除了保存着我们插入的用户数据之外，还需要保存许多额外的信息，比方说：

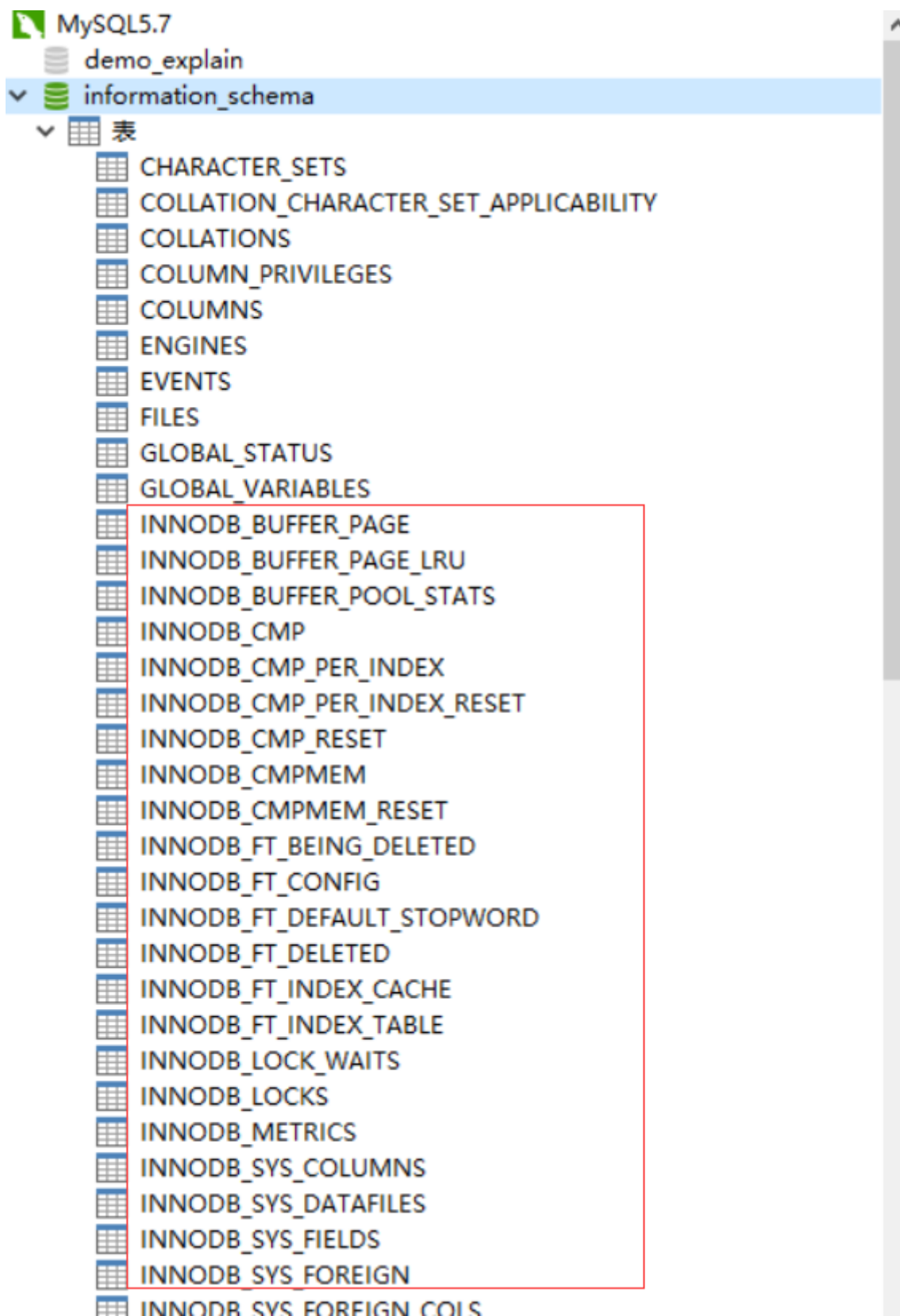
某个表属于哪个表空间，表里边有多少列，表对应的每一个列的类型是什么，该表有多少索引，每个索引对应哪几个字段，该索引对应的根页面在哪个表空间的哪个页面，该表有哪些外键，外键对应哪个表的哪些列，某个表空间对应文件系统上文件路径是什么。

上述这些数据并不是我们使用INSERT语句插入的用户数据，实际上是为了更好的管理我们这些用户数据而不得已引入的一些额外数据，这些数据也称为元数据。InnoDB存储引擎特意定义了一些列的内部系统表（internal system table）来记录这些元数据：

表名	描述
SYS_TABLES	整个InnoDB存储引擎中所有的表的信息
SYS_COLUMNS	整个InnoDB存储引擎中所有的列的信息
SYS_INDEXES	整个InnoDB存储引擎中所有的索引的信息
SYS_FIELDS	整个InnoDB存储引擎中所有的索引对应的列的信息
SYS_FOREIGN	整个InnoDB存储引擎中所有的外键的信息
SYS_FOREIGN_COLS	整个InnoDB存储引擎中所有的外键对应列的信息
SYS_TABLESPACES	整个InnoDB存储引擎中所有的表空间信息
SYS_DATAFILES	整个InnoDB存储引擎中所有的表空间对应文件系统的文件路径信息
SYS_VIRTUAL	整个InnoDB存储引擎中所有的虚拟生成列的信息

这些系统表也被称为数据字典，它们都是以B+树的形式保存在系统表空间的某些页面中，其中SYS_TABLES、SYS_COLUMNS、SYS_INDEXES、SYS_FIELDS这四个表尤其重要，称之为基本系统表。

用户是不能直接访问InnoDB的这些内部系统表的，除非你直接去解析系统表空间对应文件系统上的文件。不过InnoDB考虑到查看这些表的内容可能有助于大家分析问题，所以在系统数据库information_schema中提供了一些以innodb_sys开头的表：



在information_schema数据库中的这些以INNODB_SYS开头的表并不是真正的内部系统表（内部系统表就是我们上边唠叨的以SYS开头的那些表），而是在存储引擎启动时读取这些以SYS开头的系统表，然后填充到这些以INNODB_SYS开头的表中。

1.3.InnoDB的Buffer Pool

1.3.1.缓存的重要性

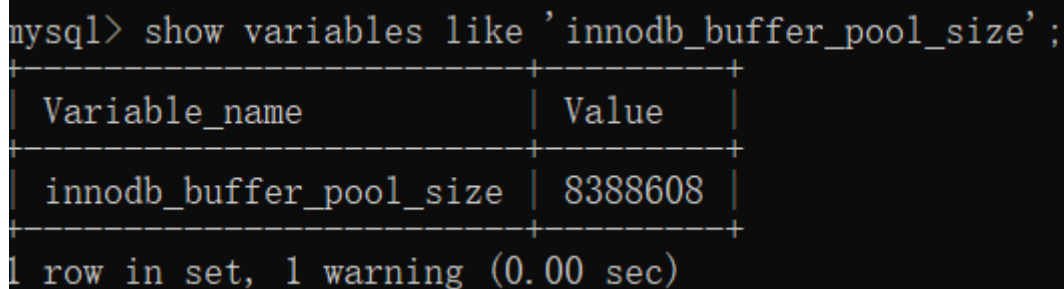
我们知道，对于使用InnoDB作为存储引擎的表来说，不管是用于存储用户数据的索引（包括聚簇索引和二级索引），还是各种系统数据，都是以页的形式存放在表空间中的，而所谓的表空间只不过是InnoDB对文件系统上一个或几个实际文件的抽象，也就是说我们的数据说到底还是存储在磁盘上的。

但是磁盘的速度慢，所以InnoDB存储引擎在处理客户端的请求时，当需要访问某个页的数据时，就会把完整的页的数据全部加载到内存中，也就是说即使我们只需要访问一个页的一条记录，那也需要先把整个页的数据加载到内存中。将整个页加载到内存中后就可以进行读写访问了，在进行完读写访问之后并不着急把该页对应的内存空间释放掉，而是将其缓存起来，这样将来有请求再次访问该页面时，就可以省去磁盘IO的开销了。

1.3.2.Buffer Pool

InnoDB为了缓存磁盘中的页，在MySQL服务器启动的时候就向操作系统申请了一片连续的内存，他们给这片内存起了个名，叫做Buffer Pool（中文名是缓冲池）。那它有多大呢？这个其实看我们机器的配置，默认情况下Buffer Pool只有8M大小，这个值其实是偏小的。

```
show variables like 'innodb_buffer_pool_size';
```



Variable_name	Value
innodb_buffer_pool_size	8388608

1 row in set, 1 warning (0.00 sec)

可以在启动服务器的时候配置innodb_buffer_pool_size参数的值，它表示BufferPool的大小，就像这样：

```
[server]
```

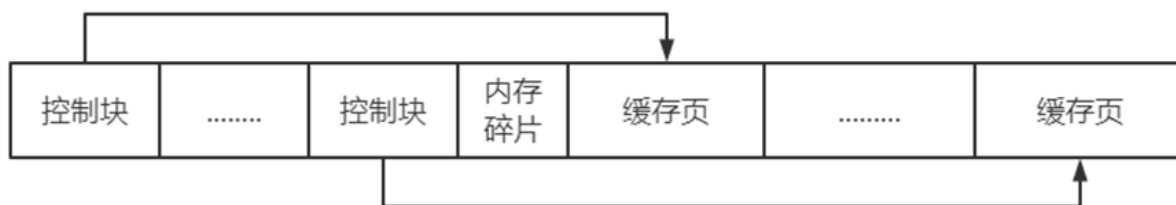
```
innodb_buffer_pool_size= 268435456
```

其中，268435456的单位是字节，也就是指定Buffer Pool的大小为256M。需要注意的是，Buffer Pool也不能太小，最小值为5M(当小于该值时会自动设置成5M)。

Buffer Pool内部组成

Buffer Pool中默认的缓存页大小和在磁盘上默认的页大小是一样的，都是16KB。为了更好的管理这些在Buffer Pool中的缓存页，InnoDB为每一个缓存页都创建了一些所谓的控制信息，这些控制信息包括该页所属的表空间编号、页号、缓存页在Buffer Pool中的地址、链表节点信息、一些锁信息以及LSN信息，当然还有一些别的控制信息。

每个缓存页对应的控制信息占用的内存大小是相同的，我们称为控制块。控制块和缓存页是一一对应的，它们都被存放到Buffer Pool中，其中控制块被存放到Buffer Pool的前边，缓存页被存放到Buffer Pool后边，所以整个Buffer Pool对应的内存空间看起来就是这样的：

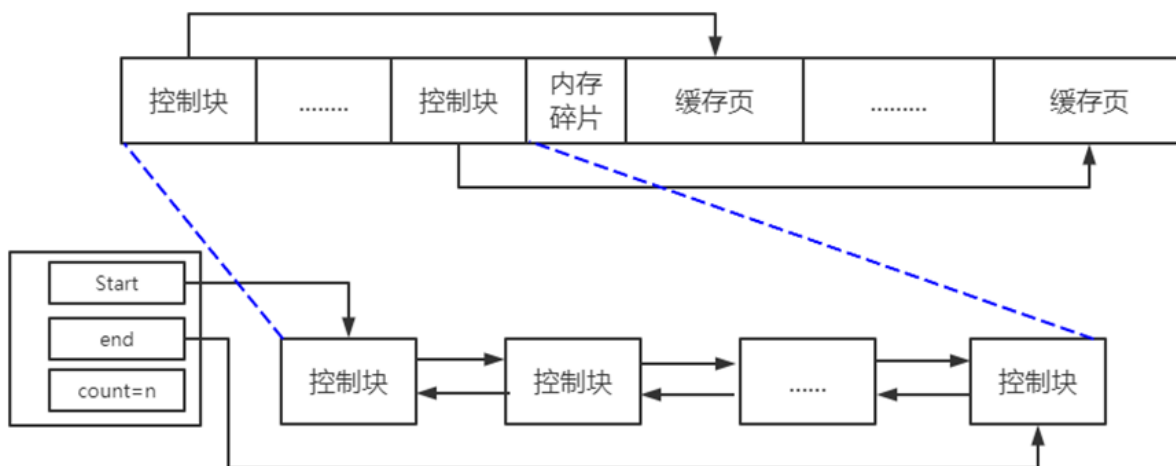


每个控制块大约占用缓存页大小的5%，而我们设置的`innodb_buffer_pool_size`并不包含这部分控制块占用的内存空间大小，也就是说InnoDB在为Buffer Pool向操作系统申请连续的内存空间时，这片连续的内存空间一般会比`innodb_buffer_pool_size`的值大5%左右。

1.3.2.1. free链表的管理

最初启动MySQL服务器的时候，需要完成对Buffer Pool的初始化过程，就是先向操作系统申请Buffer Pool的内存空间，然后把它划分成若干对控制块和缓存页。但是此时并没有真实的磁盘页被缓存到Buffer Pool中（因为还没有用到），之后随着程序的运行，会不断的有磁盘上的页被缓存到Buffer Pool中。

那么问题来了，从磁盘上读取一个页到Buffer Pool中的时候该放到哪个缓存页的位置呢？或者说怎么区分Buffer Pool中哪些缓存页是空闲的，哪些已经被使用了呢？最好在某个地方记录一下Buffer Pool中哪些缓存页是可用的，这个时候缓存页对应的控制块就派上大用场了，我们可以把所有空闲的缓存页对应的控制块作为一个节点放到一个链表中，这个链表也可以被称作free链表（或者说空闲链表）。刚刚完成初始化的Buffer Pool中所有的缓存页都是空闲的，所以每一个缓存页对应的控制块都会被加入到free链表中，假设该Buffer Pool中可容纳的缓存页数量为 n ，那增加了free链表的效果图就是这样的：



有了这个free链表之后，每当需要从磁盘中加载一个页到Buffer Pool中时，就从free链表中取一个空闲的缓存页，并且把该缓存页对应的控制块的信息填上（就是该页所在的表空间、页号之类的信息），然后把该缓存页对应的free链表节点从链表中移除，表示该缓存页已经被使用了。

1.3.2.2. 缓存页的哈希处理

我们前边说过，当我们需要访问某个页中的数据时，就会把该页从磁盘加载到Buffer Pool中，如果该页已经在Buffer Pool中的话直接使用就可以了。那么问题也就来了，我们怎么知道该页在不在Buffer Pool中呢？难不成需要依次遍历Buffer Pool中各个缓存页么？

我们其实是根据表空间号 + 页号来定位一个页的，也就相当于表空间号 + 页号是一个key，缓存页就是对应的value，怎么通过一个key来快速找着一个value呢？

所以我们可以用表空间号 + 页号作为key，缓存页作为value创建一个哈希表，在需要访问某个页的数据时，先从哈希表中根据表空间号 + 页号看看有没有对应的缓存页，如果有，直接使用该缓存页就好，如果没有，那就从free链表中选一个空闲的缓存页，然后把磁盘中对应的页加载到该缓存页的位置。

1.3.2.3. flush链表的管理

如果我们修改了Buffer Pool中某个缓存页的数据，那它就和磁盘上的页不一致了，这样的缓存页也被称为脏页（英文名：dirty page）。当然，最简单的做法就是每发生一次修改就立即同步到磁盘上对应的页上，但是频繁的往磁盘上写数据会严重的影响程序的性能。所以每次修改缓存页后，我们并不着急立即把修改同步到磁盘上，而是在未来的某个时间点进行同步。

但是如果不立即同步到磁盘的话，那之后再同步的时候我们怎么知道Buffer Pool中哪些页是脏页，哪些页从来没被修改过呢？总不能把所有的缓存页都同步到磁盘上吧，假如Buffer Pool被设置的很大，比方说300G，那一次性同步会非常慢。

所以，需要再创建一个存储脏页的链表，凡是修改过的缓存页对应的控制块都会作为一个节点加入到一个链表中，因为这个链表节点对应的缓存页都是需要被刷新到磁盘上的，所以也叫flush链表。链表的构造和free链表差不多。

1.3.2.4. LRU链表的管理

缓存不够的窘境

Buffer Pool对应的内存大小毕竟是有限的，如果需要缓存的页占用的内存大小超过了Buffer Pool大小，也就是free链表中已经有多余的空闲缓存页的时候该咋办？当然是把某些旧的缓存页从Buffer Pool中移除，然后再把新的页放进来，那么问题来了，移除哪些缓存页呢？

为了回答这个问题，我们还需要回到我们设立Buffer Pool的初衷，我们就是想减少和磁盘的IO交互，最好每次在访问某个页的时候它都已经被缓存到Buffer Pool中了。假设我们一共访问了n次页，那么被访问的页已经在缓存中的次数除以n就是所谓的缓存命中率，我们的期望就是让缓存命中率越高越好。

从这个角度出发，回想一下我们的微信聊天列表，排在前边的都是最近很频繁使用的，排在后边的自然就是最近很少使用的，假如列表能容纳下的联系人有限，你是会把最近很频繁使用的留下还是最近很少使用的留下呢？当然是留下最近很频繁使用的了。

简单的LRU链表

管理Buffer Pool的缓存页其实也是这个道理，当Buffer Pool中不再有空闲的缓存页时，就需要淘汰掉部分最近很少使用的缓存页。不过，我们怎么知道哪些缓存页最近频繁使用，哪些最近很少使用呢？

再创建一个链表，由于这个链表是为了按照最近最少使用的原则去淘汰缓存页的，所以这个链表可以被称为LRU链表（LRU的英文全称：Least Recently Used）。当我们需要访问某个页时，可以这样处理LRU链表：

如果该页不在Buffer Pool中，在把该页从磁盘加载到Buffer Pool中的缓存页时，就把该缓存页对应的控制块作为节点塞到LRU链表的头部。

如果该页已经缓存在Buffer Pool中，则直接把该页对应的控制块移动到LRU链表的头部。

也就是说：只要我们使用到某个缓存页，就把该缓存页调整到LRU链表的头部，这样LRU链表尾部就是最近最少使用的缓存页。所以当Buffer Pool中的空闲缓存页使用完时，到LRU链表的尾部找些缓存页淘汰就行了。

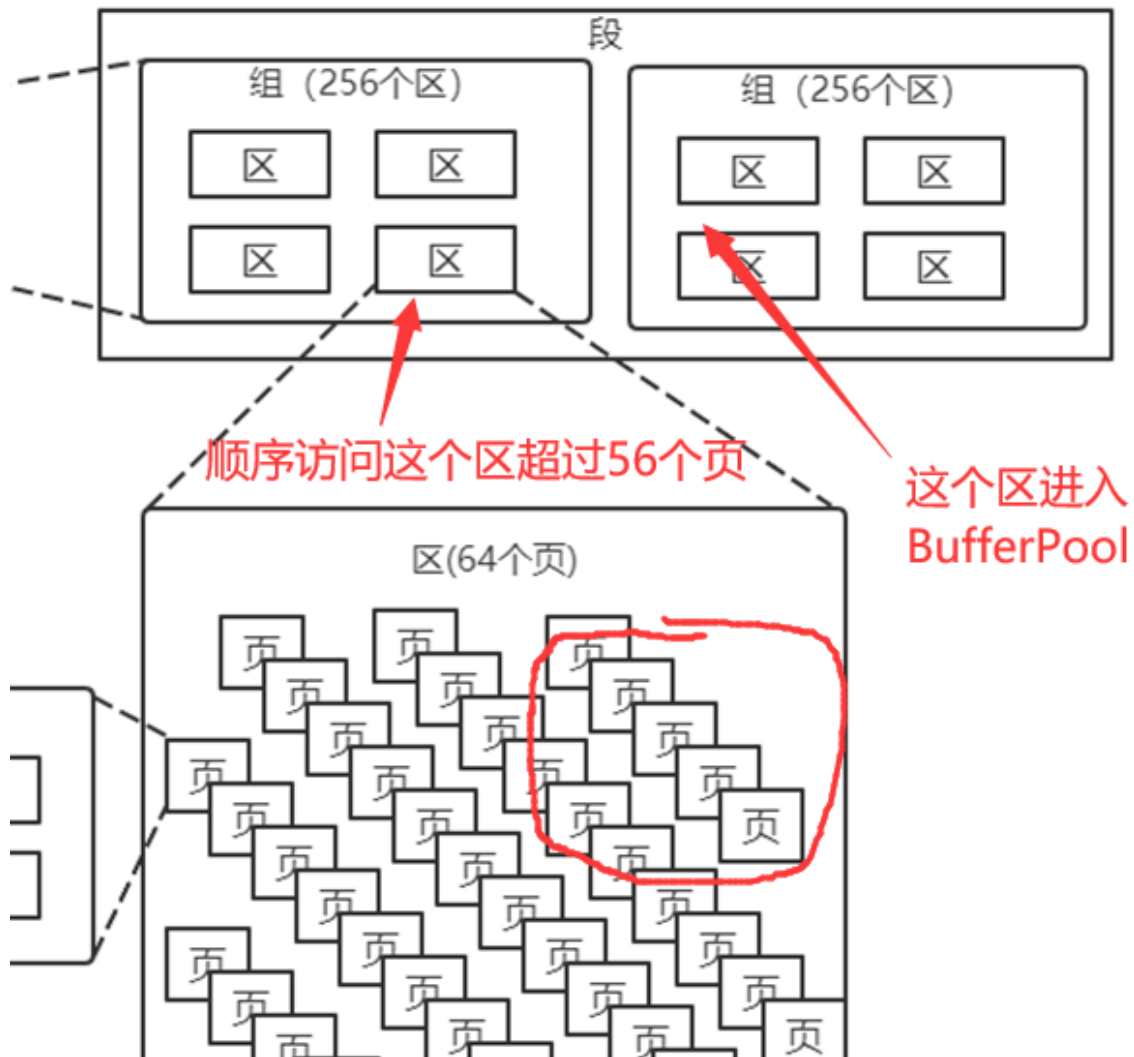
划分区域的LRU链表

但是这种实现存在两种比较尴尬的情况：

情况一：InnoDB提供了预读（英文名：readahead）。所谓预读，就是InnoDB认为执行当前的请求可能之后会读取某些页面，就预先把它们加载到Buffer Pool中。根据触发方式的不同，预读又可以细分为下边两种：

线性预读

InnoDB提供了一个系统变量`innodb_read_ahead_threshold`，如果顺序访问了某个区（extent）的页面超过这个系统变量的值，就会触发一次异步读取下一个区中全部的页面到Buffer Pool的请求。



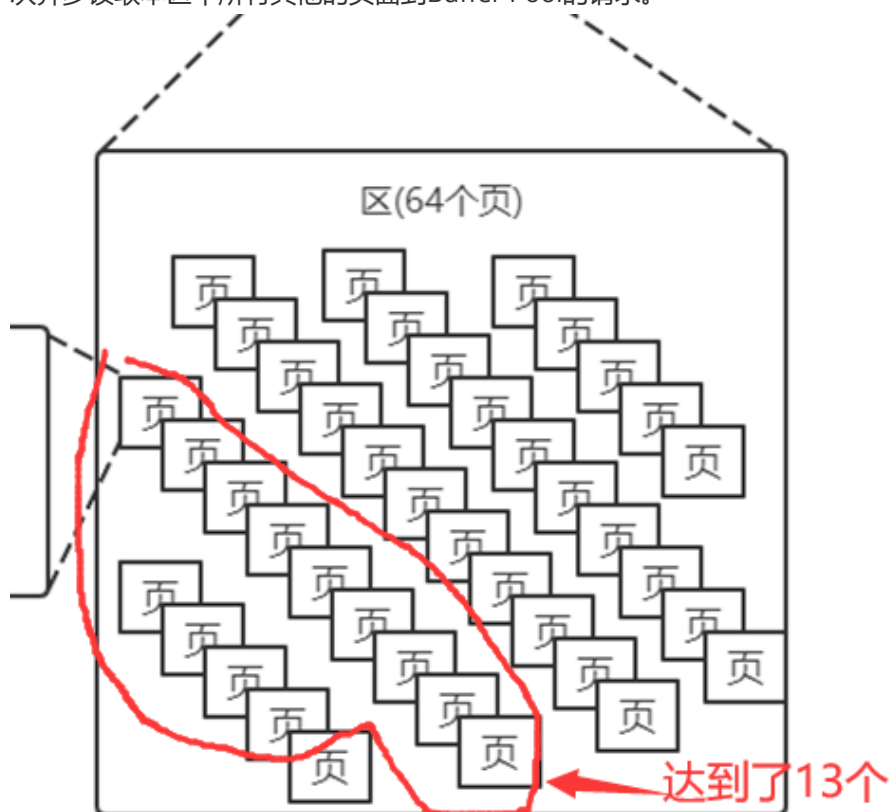
```
show variables like 'innodb_read_ahead_threshold';
```

```
mysql> show variables like 'innodb_read_ahead_threshold';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_read_ahead_threshold | 56    |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

这个`innodb_read_ahead_threshold`系统变量的值默认是56，我们可以在服务器启动时通过启动参数或者服务器运行过程中直接调整该系统变量的值。

随机预读

如果Buffer Pool中已经缓存了某个区的13个连续的页面，不论这些页面是不是顺序读取的，都会触发一次异步读取本区中所有其他的页面到Buffer Pool的请求。



如果预读到Buffer Pool中的页成功的被使用到，那就可以极大的提高语句执行的效率。可是如果用不到呢？这些预读的页都会放到LRU链表的头部，但是如果此时Buffer Pool的容量不太大而且很多预读的页面都没有用到的话，这就会导致处在LRU链表尾部的一些缓存页会很快的被淘汰掉，也就是所谓的劣币驱逐良币，会大大降低缓存命中率。

所以InnoDB同时提供了innodb_random_read_ahead系统变量，它的默认值为OFF。

```
show variables like 'innodb_random_read_ahead';
```

```
mysql> show variables like 'innodb_random_read_ahead';
```

Variable_name	Value
innodb_random_read_ahead	OFF

1 row in set, 1 warning (0.00 sec)

情况二：应用程序可能会写一些需要扫描全表的查询语句（比如没有建立合适的索引或者压根儿没有WHERE子句的查询）。

扫描全表意味着什么？意味着将访问到该表所在的所有页！假设这个表中记录非常多的话，那该表会占用特别多的页，当需要访问这些页时，会把它们统统都加载到Buffer Pool中，这也就意味着Buffer Pool中的所有页都被换了一次血，其他查询语句在执行时又得执行一次从磁盘加载到Buffer Pool的操作。而这种全表扫描的语句执行的频率也不高，每次执行都要把Buffer Pool中的缓存页换一次血，这严重的影响到其他查询对Buffer Pool的使用，从而大大降低了缓存命中率。

总结一下上边说的可能降低Buffer Pool的两种情况：

加载到Buffer Pool中的页不一定被用到。

如果非常多的使用频率偏低的页被同时加载到Buffer Pool时，可能会把那些使用频率非常高的页从Buffer Pool中淘汰掉。

因为有两种情况的存在，所以InnoDB把这个LRU链表按照一定比例分成两截，分别是：

一部分存储使用频率非常高的缓存页，所以这一部分链表也叫做热数据，或者称young区域。

另一部分存储使用频率不是很高的缓存页，所以这一部分链表也叫做冷数据，或者称old区域。

我们是按照某个比例将LRU链表分成两半的，不是某些节点固定是young区域的，某些节点固定是old区域的，随着程序的运行，某个节点所属的区域也可能发生变化。那这个划分成两截的比例怎么确定呢？对于InnoDB存储引擎来说，我们可以通过查看系统变量innodb_old_blocks_pct的值来确定old区域在LRU链表中所占的比例，比方说这样：

```
SHOW VARIABLES LIKE 'innodb_old_blocks_pct';
```

```
mysql> SHOW VARIABLES LIKE 'innodb_old_blocks_pct';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_old_blocks_pct | 37    |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

从结果可以看出来，默认情况下，old区域在LRU链表中所占的比例是37%，也就是说old区域大约占LRU链表的3/8。这个比例我们是可以设置的，我们可以在启动时修改innodb_old_blocks_pct参数来控制old区域在LRU链表中所占的比例。在服务器运行期间，我们也可以修改这个系统变量的值，不过需要注意的是，这个系统变量属于全局变量。

有了这个被划分成young和old区域的LRU链表之后，InnoDB就可以针对我们上边提到的两种可能降低缓存命中率的情况进行优化了：

针对预读的页面可能不进行后续访问情况的优化：

InnoDB规定，当磁盘上的某个页面在初次加载到Buffer Pool中的某个缓存页时，该缓存页对应的控制块会被放到old区域的头部。这样针对预读到Buffer Pool却不进行后续访问的页面就会被逐渐从old区域逐出，而不会影响young区域中被使用比较频繁的缓存页。

针对全表扫描时，短时间内访问大量使用频率非常低的页面情况的优化：

在进行全表扫描时，虽然首次被加载到Buffer Pool的页被放到了old区域的头部，但是后续会被马上访问到，每次进行访问的时候又会把该页放到young区域的头部，这样仍然会把那些使用频率比较高的页面给顶下去。

有同学会想：可不可以第一次访问该页面时不将其从old区域移动到young区域的头部，后续访问时再将其移动到young区域的头部。回答是：行不通！因为InnoDB规定每次去页面中读取一条记录时，都算是访问一次页面，而一个页面中可能会包含很多条记录，也就是说读取完某个页面的记录就相当于访问了这个页面好多次。

全表扫描有一个特点，那就是它的执行频率非常低，出现了全表扫描的语句也是我们应该尽快优化的对象。而且在执行全表扫描的过程中，即使某个页面中有很多条记录，也就是去多次访问这个页面所花费的时间也是非常少的。

所以在对某个处在old区域的缓存页进行第一次访问时就在它对应的控制块中记录下来这个访问时间，如果后续的访问时间与第一次访问的时间在某个时间间隔内，那么该页面就不会被从old区域移动到young区域的头部，否则将它移动到young区域的头部。上述的这个间隔时间是由系统变量innodb_old_blocks_time控制的：

```
SHOW VARIABLES LIKE 'innodb_old_blocks_time';
```

```
mysql> SHOW VARIABLES LIKE 'innodb_old_blocks_time';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_old_blocks_time | 1000 |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

这个innodb_old_blocks_time的默认值是1000，它的单位是毫秒，也就意味着对于从磁盘上被加载到LRU链表的old区域的某个页来说，如果第一次和最后一次访问该页面的时间间隔小于1s（很明显在一次全表扫描的过程中，多次访问一个页面中的时间不会超过1s），那么该页是会被加入到young区域的，当然，像innodb_old_blocks_pct一样，我们也可以在服务器启动或运行时设置innodb_old_blocks_time的值，这里需要注意的是，如果我们将innodb_old_blocks_time的值设置为0，那么每次我们访问一个页面时就会把该页面放到young区域的头部。

综上所述，正是因为将LRU链表划分为young和old区域这两个部分，又添加了innodb_old_blocks_time这个系统变量，才使得预读机制和全表扫描造成的缓存命中率降低的问题得到了遏制，因为用不到的预读页面以及全表扫描的页面都只会被放到old区域，而不影响young区域中的缓存页。

更进一步优化LRU链表

对于young区域的缓存页来说，我们每次访问一个缓存页就要把它移动到LRU链表的头部，这样开销是不是太大？

毕竟在young区域的缓存页都是热点数据，也就是可能被经常访问的，这样频繁的对LRU链表进行节点移动操作也会拖慢速度？为了解决这个问题，MySQL中还有一些优化策略，比如只有被访问的缓存页位于young区域的1/4的后边，才会被移动到LRU链表头部，这样就可以降低调整LRU链表的频率，从而提升性能。

1.3.3.多个Buffer Pool实例

我们上边说过，Buffer Pool本质是InnoDB向操作系统申请的一块连续的内存空间，在多线程环境下，访问Buffer Pool中的各种链表都需要加锁处理，在Buffer Pool特别大而且多线程并发访问特别高的情况下，单一的Buffer Pool可能会影响请求的处理速度。所以在Buffer Pool特别大的时候，我们可以把它们拆分成若干个小的Buffer Pool，每个Buffer Pool都称为一个实例，它们都是独立的，独立的去申请内存空间，独立的管理各种链表，所以在多线程并发访问时并不会相互影响，从而提高并发处理能力。

我们可以在服务器启动的时候通过设置innodb_buffer_pool_instances的值来修改Buffer Pool实例的个数

那每个Buffer Pool实例实际占多少内存空间呢？其实使用这个公式算出来的：

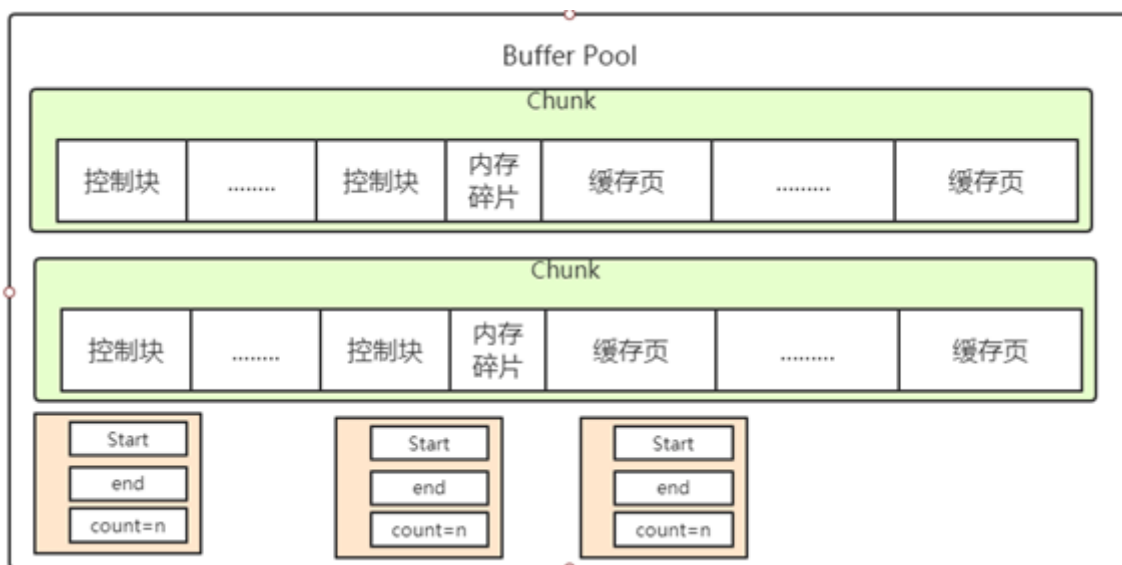
innodb_buffer_pool_size/innodb_buffer_pool_instances

也就是总共的大小除以实例的个数，结果就是每个Buffer Pool实例占用的大小。

不过也不是说Buffer Pool实例创建的越多越好，分别管理各个Buffer Pool也是需要性能开销的，InnoDB规定：当innodb_buffer_pool_size（默认128M）的值小于1G的时候设置多个实例是无效的，InnoDB会默认把innodb_buffer_pool_instances 的值修改为1。所以Buffer Pool大于或等于1G的时候设置应该多个Buffer Pool实例。

1.3.4. innodb_buffer_pool_chunk_size

在MySQL 5.7.5之前，Buffer Pool的大小只能在服务器启动时通过配置innodb_buffer_pool_size启动参数来调整大小，在服务器运行过程中是不允许调整该值的。不过MySQL在5.7.5以及之后的版本中支持了在服务器运行过程中调整Buffer Pool大小的功能，但是有一个问题，就是每次当我们要重新调整Buffer Pool大小时，都需要重新向操作系统申请一块连续的内存空间，然后将旧的Buffer Pool中的内容复制到这一块新空间，这是极其耗时的。所以MySQL决定不再一次性为某Buffer Pool实例向操作系统申请一大片连续的内存空间，而是以一个所谓的chunk为单位向操作系统申请空间。也就是说一个Buffer Pool实例其实是由若干个chunk组成的，一个chunk就代表一片连续的内存空间，里边儿包含了若干缓存页与其对应的控制块：



正是因为发明了这个chunk的概念，我们在服务器运行期间调整Buffer Pool的大小时就是以chunk为单位增加或者删除内存空间，而不需要重新向操作系统申请一片大的内存，然后进行缓存页的复制。这个所谓的chunk的大小是我们在启动操作MySQL服务器时通过innodb_buffer_pool_chunk_size启动参数指定的，它的默认值是134217728，也就是128M。不过需要注意的是，innodb_buffer_pool_chunk_size的值只能在服务器启动时指定，在服务器运行过程中是不可以修改的。

```
SHOW VARIABLES LIKE 'innodb_buffer_pool_chunk_size';
```

```
mysql> SHOW VARIABLES LIKE 'innodb_buffer_pool_chunk_size';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| innodb_buffer_pool_chunk_size | 8388608 |
+-----+-----+
1 row in set, 1 warning (0.00 sec)
```

Buffer Pool的缓存页除了用来缓存磁盘上的页面以外，还可以存储锁信息、自适应哈希索引等信息。

1.3.5. 查看Buffer Pool的状态信息

MySQL给我们提供了SHOW ENGINE INNODB STATUS语句来查看关于InnoDB存储引擎运行过程中的一些状态信息，其中就包括Buffer Pool的一些信息，我们看一下（为了突出重点，我们只把输出中关于Buffer Pool的部分提取了出来）：

```
SHOW ENGINE INNODB STATUS\G
```

这里边的每个值都代表什么意思如下，知道即可：

```
-----
BUFFER POOL AND MEMORY
-----
Total large memory allocated 8585216
Dictionary memory allocated 394405
Buffer pool size      512
Free buffers          256
Database pages        256
Old database pages    0
Modified db pages     0
Pending reads         0
Pending writes: LRU 0, flush list 0, single page 0
Pages made young 0, not young 0
0.00 youngs/s, 0.00 non-youngs/s
Pages read 394, created 53, written 78
0.00 reads/s, 0.00 creates/s, 0.00 writes/s
No buffer pool page gets since the last printout
Pages read ahead 0.00/s, evicted without access 0.00/s, Random read ahead 0.00/s
LRU len: 256, unzip_LRU len: 0
I/O sum[0]:cur[0], unzip sum[0]:cur[0]
```

Total large memory allocated

代表Buffer Pool向操作系统申请的连续内存空间大小，包括全部控制块、缓存页、以及碎片的大小。

Dictionary memory allocated

为数据字典信息分配的内存空间大小，注意这个内存空间和Buffer Pool没啥关系，不包括在Total memory allocated中。

Buffer pool size

代表该Buffer Pool可以容纳多少缓存页，注意，单位是页！

Free buffers

代表当前Buffer Pool还有多少空闲缓存页，也就是free链表中还有多少个节点。

Database pages

代表LRU链表中的页的数量，包含young和old两个区域的节点数量。

Old database pages

代表LRU链表old区域的节点数量。

Modified db pages

代表脏页数量，也就是flush链表中节点的数量。

Pending reads

正在等待从磁盘上加载到Buffer Pool中的页面数量。

当准备从磁盘中加载某个页面时，会先为这个页面在Buffer Pool中分配一个缓存页以及它对应的控制块，然后把这个控制块添加到LRU的old区域的头部，但是这个时候真正的磁盘页并没有被加载进来，Pending reads的值会跟着加1。

Pending writes

LRU：即将从LRU链表中刷新到磁盘中的页面数量。

Pending writes

flush list：即将从flush链表中刷新到磁盘中的页面数量。

Pending writes

single page：即将以单个页面的形式刷新到磁盘中的页面数量。

Pages made young

代表LRU链表中曾经从old区域移动到young区域头部的节点数量。

Page made not young

在将innodb_old_blocks_time设置的值大于0时，首次访问或者后续访问某个处在old区域的节点时由于不符合时间间隔的限制而不能将其移动到young区域头部时，Page made not young的值会加1。

young/s：代表每秒从old区域被移动到young区域头部的节点数量。

non-young/s：代表每秒由于不满足时间限制而不能从old区域移动到young区域头部的节点数量。

Pages read、created、written：代表读取，创建，写入了多少页。后边跟着读取、创建、写入的速率。

Buffer pool hit rate：

表示在过去某段时间，平均访问1000次页面，有多少次该页面已经被缓存到Buffer Pool了。

young-making rate：

表示在过去某段时间，平均访问1000次页面，有多少次访问使页面移动到young区域的头部了。

not(young-making rate)：

表示在过去某段时间，平均访问1000次页面，有多少次访问没有使页面移动到young区域的头部。

LRU len：代表LRU链表中节点的数量。

unzip_LRU：代表unzip_LRU链表中节点的数量。

I/O sum：最近50s读取磁盘页的总数。

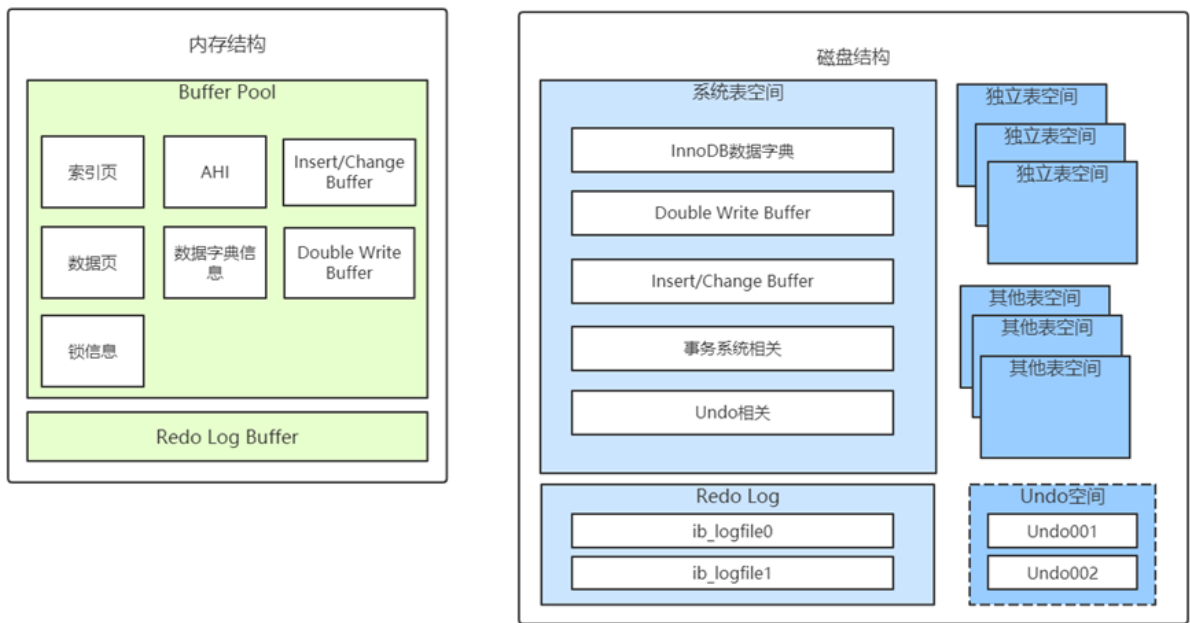
I/O cur：现在正在读取的磁盘页数量。

I/O unzip sum：最近50s解压的页面数量。

I/O unzip cur：正在解压的页面数量。

1.3.3.InnoDB的内存结构总结

InnoDB的内存结构和磁盘存储结构图总结如下：



其中的Insert/Change Buffer主要是用于对二级索引的写入优化，Undo空间则是undo日志一般放在系统表空间，但是通过参数配置后，也可以用独立表空间存放，所以用虚线表示。